SERIES 11

github.com/madhavan-21/ebpf-mastery

TC Traffic Classifier & Rate Limiting

Lesson 11: Egress Packet Shaping with Traffic Control eBPF Hooks

Madhavan S
SDE - Atatus

Date: July 21, 2026
Version: v1.0.0

Table of Contents

1. Beginner Level

   Q1. What is the difference between XDP and TC hooks? When should you use each?

   Q2. What is the clsact qdisc and why is it required for eBPF TC programs?

   Q3. What does TC_ACT_SHOT do compared to TC_ACT_OK?

2. Intermediate Level

   Q4. How does the TC eBPF context __sk_buff differ from XDP's xdp_md?

   Q5. Why must you use clsact instead of attaching to an existing qdisc like htb?

   Q6. How would you implement per-pod rate limiting in Kubernetes using TC eBPF?

3. Advanced Level

   Q7. Explain the packet data access model in TC programs. What is "direct access" vs "helper access"?

   Q8. How can TC programs modify packets in-flight? What helpers are available?

   Q9. What happens if you attach both an XDP program and a TC ingress program to the same interface?

   Q10. How does Cilium handle atomic policy updates in TC without disrupting active connections?

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

   Task 1: Explore TC Qdisc and Filters

   Task 2: Compare TC Return Codes

   Task 3: Monitor with bpftool

2. Coding Assignment: Bandwidth-Based Rate Limiting

3. Advanced Extension: Per-Destination Rate Limiting

Lesson 11: TC Traffic Classifier & Rate Limiting

1. Lesson Overview

In this lesson, you will build a **production-grade traffic classifier and rate limiter** using the Linux Traffic Control (TC) hook. TC is the eBPF attachment point used by Cilium for all egress policy enforcement, and by Meta's Katran for outbound traffic shaping. Unlike XDP which operates at ingress only, TC programs can classify, rate-limit, and redirect packets on **both ingress and egress** paths. We will attach an eBPF program to the TC egress hook to enforce per-IP bandwidth limits by counting packets per source IP and dropping traffic that exceeds a configured threshold.

---

2. Foundations & Core Technical Concepts

What is Traffic Control (TC)?

The Linux **Traffic Control** subsystem is the kernel's packet scheduling and shaping framework. It sits at the queueing discipline (qdisc) layer — after the kernel networking stack has processed a packet but before it is handed to the NIC driver for transmission.

TC has been part of the Linux kernel since version 2.2, but eBPF integration (via `BPF_PROG_TYPE_SCHED_CLS`) was added in kernel 4.1. This allows you to write custom packet classifiers entirely in eBPF, replacing complex `tc filter` rules with a single compiled program.

TC vs XDP: When to Use Which

- **XDP** fires at the NIC driver level, before any `sk_buff` allocation. It is the fastest path for **ingress-only** filtering (DDoS mitigation). XDP cannot see egress packets.
- **TC** fires after `sk_buff` creation and has access to the full networking stack context. It works on **both ingress and egress**, making it essential for outbound policy enforcement, NAT, and traffic shaping.

TC Hook Points

```
Ingress Path:   NIC → XDP → TC ingress → Netfilter → Socket
Egress Path:    Socket → Netfilter → TC egress → NIC Driver
```

TC Return Codes

Code	Value	Meaning
`TC_ACT_OK`	0	Accept the packet, continue processing
`TC_ACT_SHOT`	2	Drop the packet immediately
`TC_ACT_REDIRECT`	7	Redirect packet to another interface
`TC_ACT_PIPE`	3	Pass to next filter in the chain

What is a Qdisc?

A **queueing discipline** (qdisc) is the kernel data structure that manages how packets are queued for transmission. The `clsact` qdisc is a special TC qdisc that allows attaching eBPF classifiers to both ingress and egress without affecting the default packet scheduling behavior.

---

3. Problem Statement & Real-World Use Case

The Problem: Noisy Neighbor in Multi-Tenant Networks

In a Kubernetes cluster, a single pod running a data pipeline can saturate the node's egress bandwidth, starving other pods of network capacity. Traditional `tc` rules using `htb` (hierarchical token bucket) require static configuration per-pod and cannot adapt to dynamic workloads.

Real Production Story: Cilium Bandwidth Manager

Cilium uses TC eBPF programs to enforce per-pod bandwidth limits in Kubernetes. When a pod's `kubernetes.io/egress-bandwidth` annotation is set, Cilium attaches a TC egress classifier that counts bytes per connection and drops packets exceeding the rate limit — all without any iptables rules or kernel module changes.

Meta's Katran also uses TC hooks for outbound health check traffic classification, ensuring that backend health probes never compete with production traffic.

---

4. TC vs Iptables for Traffic Shaping

Feature	iptables + HTB	eBPF TC Classifier
Attachment	Netfilter hooks	TC qdisc layer
Egress support	Via POSTROUTING chain	Native TC egress
Per-flow logic	Limited to match rules	Full programmability
Update mechanism	Rule reload disrupts traffic	Atomic map updates
Performance	Linear rule scan	O(1) map lookup

---

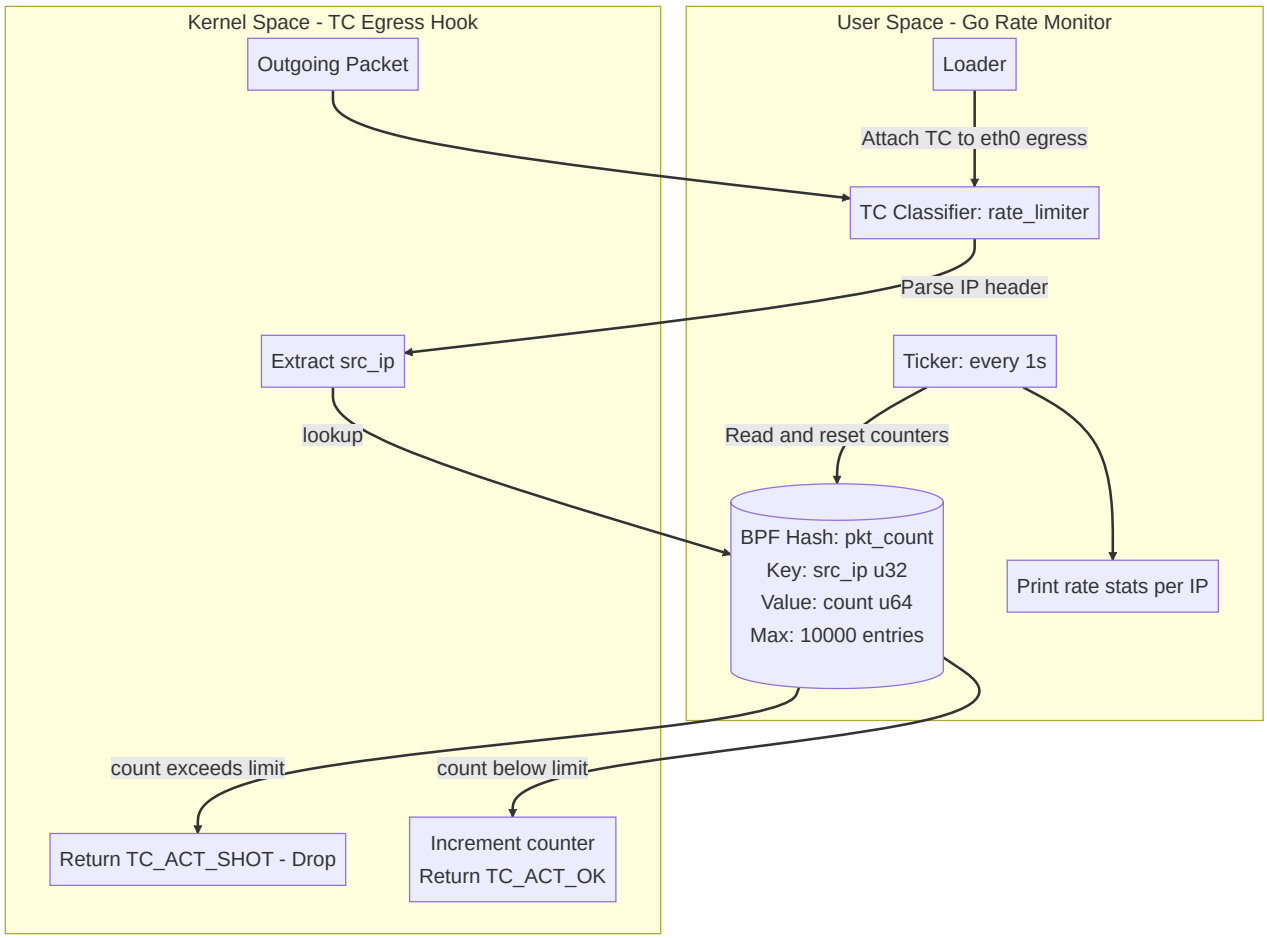
5. Internal Kernel Flow

When a packet is transmitted from a socket on a TC-enabled interface:

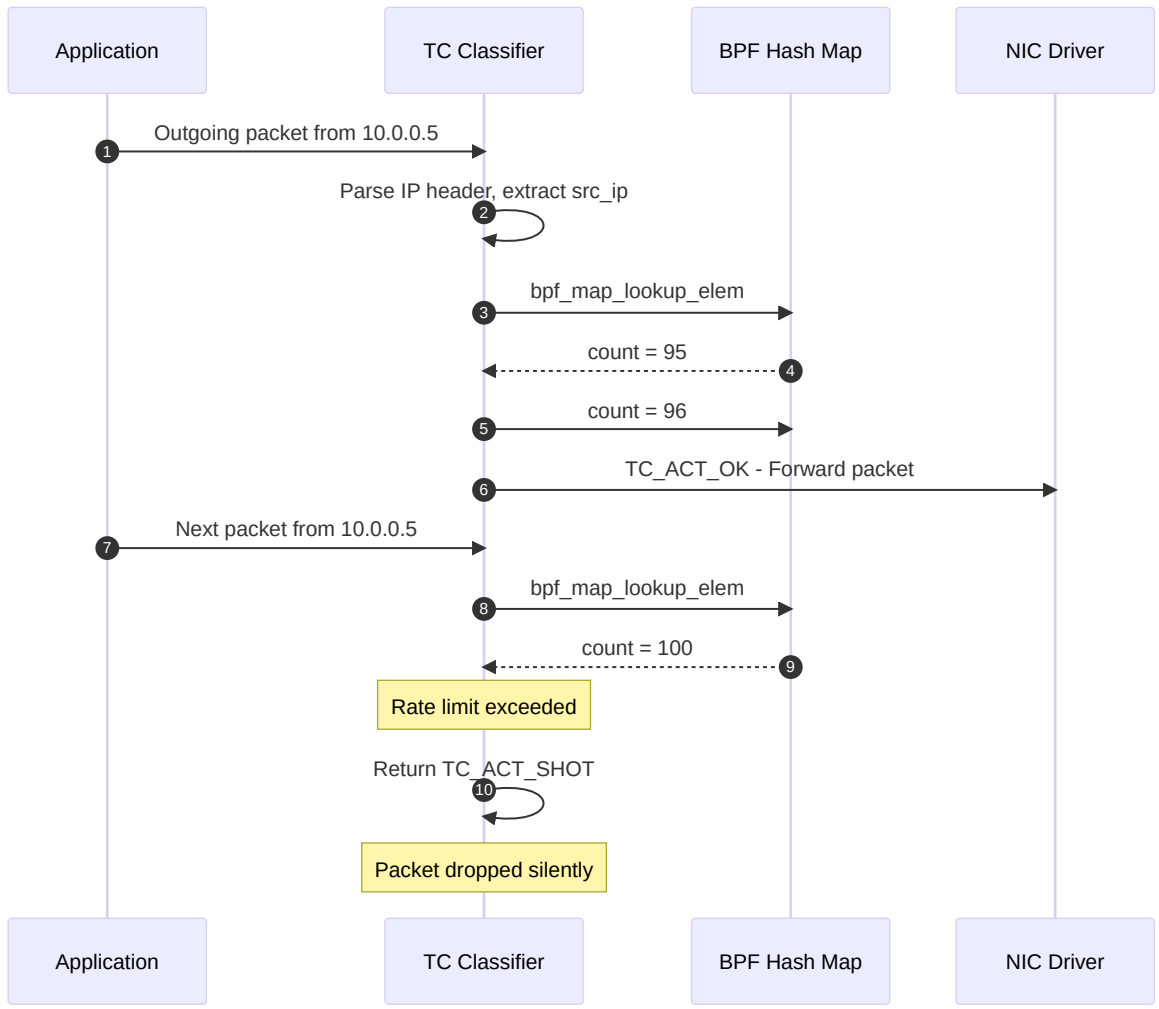
1. The kernel networking stack constructs an `sk_buff` and routes it to the egress interface.
2. The `clsact` qdisc intercepts the packet before NIC driver submission.
3. Our TC eBPF program receives the `__sk_buff *skb` context pointer.
4. We parse the IP header to extract the source IP address.
5. We look up the per-IP packet counter in our BPF hash map.
6. If the counter exceeds the rate limit threshold, we return `TC_ACT_SHOT` to drop the packet.
7. Otherwise, we increment the counter and return `TC_ACT_OK` to allow transmission.

---

6. Architecture Diagram



7. Rate Limiting Sequence



8. Data Structure Layout

```
Rate Limiter Map Entry:

Key: __u32 src_ip (4 bytes)
+--------------------+
|  src_ip (4 bytes)|
+--------------------+

Value: __u64 pkt_count (8 bytes)
+----------------------------+
|  pkt_count (8 bytes)    |
+----------------------------+

BPF Hash Map: pkt_count
max_entries: 10,000
Memory: ~10,000 x (4 + 8 + hash overhead) ≈ 500KB
```

---

9. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

```c
// Series 11: TC Traffic Classifier & Rate Limiter
#include <linux/bpf.h>
#include <linux/pkt_cls.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

#define RATE_LIMIT 100  // Max packets per interval per source IP

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 10000);
    __type(key, __u32);
    __type(value, __u64);
} pkt_count SEC(".maps");

SEC("tc")
int rate_limiter(struct __sk_buff *skb)
{
    void *data     = (void *)(long)skb->data;
    void *data_end = (void *)(long)skb->data_end;

    // Parse Ethernet header
    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)
        return TC_ACT_OK;
    if (eth->h_proto != bpf_htons(ETH_P_IP))
        return TC_ACT_OK;

    // Parse IP header
    struct iphdr *iph = (void *)(eth + 1);
    if ((void *)(iph + 1) > data_end)
        return TC_ACT_OK;

    __u32 src_ip = iph->saddr;

    // Lookup current packet count for this source IP
    __u64 *count = bpf_map_lookup_elem(&pkt_count, &src_ip);
    if (count) {
```

```
        if (*count >= RATE_LIMIT) {
            // Rate limit exceeded — drop the packet
            return TC_ACT_SHOT;
        }
        __sync_fetch_and_add(count, 1);
    } else {
        // First packet from this IP — initialize counter
        __u64 init_val = 1;
        bpf_map_update_elem(&pkt_count, &src_ip, &init_val, BPF_ANY);
    }

    return TC_ACT_OK;
}

char LICENSE[] SEC("license") = "GPL";
```

The Go Loader (`main.go`)

```go
package main

import (
        "encoding/binary"
        "fmt"
        "log"
        "net"
        "os"
        "os/exec"
        "os/signal"
        "syscall"
        "time"

        "github.com/cilium/ebpf"
)

func main() {
        log.Println("═══════════════════════════════════════════════════")
        log.Println("  eBPF Mastery: Lesson 11 — TC Rate Limiter      ")
        log.Println("═══════════════════════════════════════════════════")

        spec, err := ebpf.LoadCollectionSpec("main.bpf.o")
        if err ≠ nil {
                log.Fatalf("Load spec: %v", err)
        }
        coll, err := ebpf.NewCollection(spec)
        if err ≠ nil {
                log.Fatalf("Collection: %v", err)
        }
        defer coll.Close()

        prog := coll.Programs["rate_limiter"]
        pktMap := coll.Maps["pkt_count"]

        iface := "lo"

        // Setup clsact qdisc and attach TC program
        exec.Command("tc", "qdisc", "del", "dev", iface, "clsact").Run()
        if out, err := exec.Command("tc", "qdisc", "add", "dev", iface, "clsact").Cc
                log.Fatalf("tc qdisc add: %s %v", out, err)
        }

        // Pin the program and attach via tc filter
        pinPath := "/sys/fs/bpf/rate_limiter"
```

```
        os.Remove(pinPath)
        if err := prog.Pin(pinPath); err ≠ nil {
                log.Fatalf("Pin program: %v", err)
        }
        defer os.Remove(pinPath)

        fd := prog.FD()
        cmd := exec.Command("tc", "filter", "add", "dev", iface, "egress",
                "bpf", "da", "fd", fmt.Sprintf("%d", fd), "name", "rate_limiter")
        if out, err := cmd.CombinedOutput(); err ≠ nil {
                log.Fatalf("tc filter add: %s %v", out, err)
        }
        defer exec.Command("tc", "qdisc", "del", "dev", iface, "clsact").Run()

        log.Printf("TC rate limiter attached to %s egress. Limit: 100 pkts/interval\

        sigChan := make(chan os.Signal, 1)
        signal.Notify(sigChan, os.Interrupt, syscall.SIGTERM)
        ticker := time.NewTicker(2 * time.Second)
        defer ticker.Stop()

        for {
                select {
                case ←ticker.C:
                        printStats(pktMap)
                        resetCounters(pktMap)
                case ←sigChan:
                        fmt.Println("\nDetaching TC program. Goodbye!")
                        return
                }
        }
}

func printStats(m *ebpf.Map) {
        fmt.Println("\n--- Egress Rate Stats ---")
        var key uint32
        var val uint64
        iter := m.Iterate()
        for iter.Next(&key, &val) {
                ip := make(net.IP, 4)
                binary.LittleEndian.PutUint32(ip, key)
                fmt.Printf("  %s: %d packets\n", ip.String(), val)
        }
}

func resetCounters(m *ebpf.Map) {
        var key uint32
```

```
        var keys []uint32
        var val uint64
        iter := m.Iterate()
        for iter.Next(&key, &val) {
                keys = append(keys, key)
        }
        for _, k := range keys {
                m.Delete(&k)
        }
}
```

---

10. Running the Lab

Phase 1: Compile and Load

```
cd series/series-11/source-code
make
sudo ./loader
```

Phase 2: Generate Traffic

```
# In another terminal, flood with packets
for i in $(seq 1 200); do curl -s http://localhost:8080/ > /dev/null 2>&1; done
```

Phase 3: Observe Rate Limiting

```
--- Egress Rate Stats ---
  127.0.0.1: 100 packets
  # Remaining 100 packets were dropped by TC_ACT_SHOT
```

---

11. bpftool Debugging

```
# List TC-attached programs
sudo tc filter show dev lo egress

# Inspect the map contents
sudo bpftool map list | grep pkt_count
sudo bpftool map dump id <ID>
```

---

12. Common Mistakes & Verifier Errors

Error: TC program attached but no packets flow

If you forget to add the `clsact` qdisc before attaching the filter, the TC program will silently fail to receive packets. Always run `tc qdisc add dev <iface> clsact` first.

Error: Verifier rejects sk_buff direct access

TC programs access packet data through `skb->data` and `skb->data_end`. If your bounds check is missing or incorrect, the verifier will reject the program with `invalid access to packet`.

---

13. Performance Analysis

Metric	iptables rate limiting	eBPF TC Rate Limiter
Lookup	Linear rule scan	O(1) hash map lookup
Update	Reload entire ruleset	Atomic per-entry update
Egress support	Via POSTROUTING	Native TC egress hook
Memory	Per-rule kernel memory	Bounded map allocation

---

14. Common Mistakes

- **Forgetting clsact qdisc**: TC eBPF programs require the `clsact` qdisc to be attached to the interface before any filters can be added.
- **Using XDP return codes in TC**: TC programs must return `TC_ACT_OK`, `TC_ACT_SHOT`, etc. Using XDP return codes like `XDP_PASS` will cause undefined behavior.

---

15. Best Practices

- **Use Per-CPU maps for counters under high throughput**: Replace `BPF_MAP_TYPE_HASH` with `BPF_MAP_TYPE_PERCPU_HASH` to avoid atomic contention on hot paths.
- **Reset counters periodically**: Without periodic resets from user space, the rate limiter will permanently block IPs that hit the threshold once.

---

16. Summary

- TC hooks operate at the qdisc layer and support both ingress and egress packet processing.
- `TC_ACT_SHOT` drops packets, `TC_ACT_OK` forwards them — giving you full control over egress traffic.
- The `clsact` qdisc is required for attaching eBPF classifiers without disrupting existing qdiscs.
- Combining TC with BPF hash maps enables per-IP or per-flow rate limiting at kernel speed.

---

17. Further Reading

- Cilium TC Datapath Architecture
- Linux TC Man Page
- Meta Katran: TC for Health Checks

Technical Deep-Dive Q&A: TC Traffic Classification & Rate Limiting

This guide contains detailed technical questions and model answers for eBPF TC classifier concepts.

---

1. Beginner Level

Q1. What is the difference between XDP and TC hooks? When should you use each?

Answer: XDP and TC are both eBPF attachment points for packet processing, but they operate at different layers:

- **XDP** fires at the NIC driver level, before any `sk_buff` allocation. It is the fastest possible path for **ingress-only** packet filtering. XDP cannot process egress traffic.
- **TC** fires at the queueing discipline layer, after `sk_buff` creation. It supports **both ingress and egress**, making it essential for outbound traffic shaping, NAT, and policy enforcement.

Rule of thumb: Use XDP for high-speed ingress filtering (DDoS). Use TC when you need egress control or full `sk_buff` access.

Q2. What is the `clsact` qdisc and why is it required for eBPF TC programs?

Answer: `clsact` is a special queueing discipline that acts as a container for eBPF classifier programs. Unlike traditional qdiscs (like `htb` or `prio`), `clsact` does not affect packet scheduling — it only provides hook points for attaching eBPF programs on both ingress and egress.

Without `clsact`, the `tc filter add` command has no qdisc to attach the classifier to, and the command will fail silently or with an error.

Q3. What does `TC_ACT_SHOT` do compared to `TC_ACT_OK`?

Answer:

- `TC_ACT_SHOT` (value 2): Immediately drops the packet and frees the associated `sk_buff`. The packet never reaches the NIC or the socket.
- `TC_ACT_OK` (value 0): Accepts the packet and continues normal processing through the networking stack.

---

2. Intermediate Level

Q4. How does the TC eBPF context `__sk_buff` differ from XDP's `xdp_md`?

Answer: `__sk_buff` provides a much richer context than `xdp_md`:

- `__sk_buff`: Has fields for `protocol`, `ifindex`, `mark`, `priority`, `cb[]` (control buffer), and direct packet data access via `data` / `data_end`. It represents a fully constructed socket buffer.
- `xdp_md`: Only provides `data`, `data_end`, `data_meta`, `ingress_ifindex`, and `rx_queue_index`. It represents raw packet bytes before any kernel processing.

The tradeoff is performance: `sk_buff` allocation adds latency, but `__sk_buff` gives you access to routing decisions, socket information, and packet metadata that XDP cannot see.

Q5. Why must you use `clsact` instead of attaching to an existing qdisc like `htb`?

Answer: eBPF TC classifiers require a qdisc that supports the `direct-action` (da) flag. The `clsact` qdisc was specifically designed for this purpose — it allows the eBPF program to make the final verdict (accept, drop, redirect) without requiring additional TC actions.

Traditional qdiscs like `htb` use a classifier-action model where the classifier only selects a class, and a separate action module decides the fate. eBPF programs with `da` mode collapse both into a single program.

Q6. How would you implement per-pod rate limiting in Kubernetes using TC eBPF?

Answer:

1. Create a BPF hash map keyed by the pod's veth interface index (or cgroup ID).
2. Attach a TC egress classifier to each pod's veth interface using `clsact`.
3. The eBPF program looks up the byte count for the current interface and compares it against the pod's bandwidth annotation.
4. If the byte count exceeds the rate, return `TC_ACT_SHOT` to drop excess packets.

5. User space periodically resets the counters to create sliding window rate limiting.

This is exactly how Cilium's Bandwidth Manager works in production.

---

3. Advanced Level

Q7. Explain the packet data access model in TC programs. What is "direct access" vs "helper access"?

Answer: TC programs can access packet data in two ways:

- **Direct access**: Read `skb->data` and `skb->data_end` pointers, then cast them to header structs. This is fast but requires explicit bounds checks that the verifier validates.
- **Helper access**: Use `bpf_skb_load_bytes()` to copy packet bytes into a local buffer. This is slower but works when direct access is not available (e.g., fragmented packets).

Modern kernels (5.x+) support direct access for TC programs, which is preferred for performance.

Q8. How can TC programs modify packets in-flight? What helpers are available?

Answer: TC programs have full packet mutation capabilities:

- `bpf_skb_store_bytes()`: Write bytes to a specific offset in the packet.
- `bpf_l3_csum_replace()`: Update the L3 (IP) checksum after modifying IP header fields.
- `bpf_l4_csum_replace()`: Update the L4 (TCP/UDP) checksum after modifying ports.
- `bpf_skb_change_head()`: Add or remove bytes from the packet head (for encapsulation).
- `bpf_redirect()`: Redirect the packet to a different network interface.

These helpers enable NAT, encapsulation (VXLAN, GRE), and packet rewriting directly in the TC path.

Q9. What happens if you attach both an XDP program and a TC ingress program to the same interface?

Answer: Both programs execute in sequence:

1. **XDP fires first** at the driver level. If it returns `XDP_PASS`, the packet continues.
2. The kernel allocates an `sk_buff` from the packet data.

3. **TC ingress fires** with the constructed `sk_buff`.

This means XDP can pre-filter traffic (dropping DDoS) before TC ingress does fine-grained classification. Packets dropped by XDP never reach TC, saving the cost of `sk_buff` allocation.

Q10. How does Cilium handle atomic policy updates in TC without disrupting active connections?

Answer: Cilium uses BPF map updates rather than program replacement:

1. Policy rules are stored in BPF maps (hash maps keyed by identity or IP).
2. When a policy changes, Cilium updates the map entries atomically.
3. The TC program itself is never replaced — it reads the current policy from the map on each packet.
4. For program logic changes, Cilium uses `bpf_prog_replace()` which atomically swaps the old program for the new one without any packet loss.

This atomic update model ensures zero-downtime policy enforcement.

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

Task 1: Explore TC Qdisc and Filters

Inspect the current TC configuration on your system:

```
# List all qdiscs on all interfaces
tc qdisc show

# Add clsact qdisc to loopback
sudo tc qdisc add dev lo clsact

# List filters on egress
sudo tc filter show dev lo egress

# Clean up
sudo tc qdisc del dev lo clsact
```

- What is the default qdisc on your system? (hint: check `tc qdisc show dev eth0`)
- What happens if you try to add a filter without first adding a clsact qdisc?

Task 2: Compare TC Return Codes

Modify the eBPF program to use different return codes and observe the behavior:

1. Return `TC_ACT_OK` for all packets — verify all traffic flows normally.
2. Return `TC_ACT_SHOT` for all packets — verify all egress traffic is blocked.
3. Return `TC_ACT_PIPE` — research what this does in a multi-filter chain.

Task 3: Monitor with bpftool

After loading the TC program:

```
# List all loaded BPF programs
sudo bpftool prog list

# Find the TC classifier program
sudo bpftool prog show name rate_limiter

# Dump the BPF bytecode
sudo bpftool prog dump xlated name rate_limiter
```

- How many BPF instructions does the program use?
- What is the program's memory footprint?

---

2. Coding Assignment: Bandwidth-Based Rate Limiting

Objective: Extend the rate limiter to enforce byte-based bandwidth limits instead of packet counts.

Requirements:

1. Change the map value from packet count to byte count.
2. Track `iph->tot_len` (total IP packet length) instead of incrementing by 1.
3. Set a bandwidth limit of 1MB per source IP per interval.
4. Log which IPs are being rate-limited in user space.

Hints:

- Use `bpf_ntohs(iph->tot_len)` to get the packet size in host byte order.
- 1MB = 1,048,576 bytes. Compare the accumulated byte count against this threshold.

---

3. Advanced Extension: Per-Destination Rate Limiting

Objective: Implement rate limiting based on destination IP instead of source IP.

Requirements:

1. Use `iph->daddr` instead of `iph->saddr` as the map key.

2. This models the "outbound connection flood" scenario where a compromised host tries to contact many external IPs.
3. Add a configurable allowlist map that exempts certain destination IPs from rate limiting.